



SMART INDIA HACKATHON 2026

V-CAPAT PRO

**Virtual Camera Acquisition, Pointing & Tracking System
for Mobile Free Space Optical Communication Terminals**

TECHNICAL REPORT - VERSION 1.1

Problem Statement ID: SIH26169

Organization: Department of Space / Indian Space Research Organisation

Category: Software | Theme: Smart Automation / Space Technology

Team: NeXGen Coder's | Team ID: 160644

Report date: 02 October 2026

Scope: software coarse alignment for optical beacon acquisition and tracking. This prototype is a research/hackathon testbed and is not a flight-qualified optical terminal.

1. Executive Summary and Problem Understanding

Free Space Optical Communication (FSOC) uses highly directional optical beams. Before fine pointing can take over, the coarse Pointing, Acquisition and Tracking (PAT) stage must observe the surrounding scene, acquire a remote optical beacon, estimate its position, and continuously reposition a camera or pointing mechanism so that the remote terminal remains inside the field of view.

The principal development barrier is that validating PAT algorithms directly on precision cameras, pan-tilt mechanisms and optical hardware is expensive and difficult to reproduce. V-CAPAT PRO addresses the software-development stage by providing a configurable virtual scene, movable virtual camera, synthetic optical beacon targets, disturbances, automatic detection, state estimation, closed-loop control, benchmark-video processing and measurable performance evidence.

1.1 Functional objective

- Autonomously detect and identify a designated moving optical beacon from image data.
- Estimate target position and short-term motion while avoiding use of simulator ground truth in the tracking/control loop.
- Continuously command virtual pan and tilt so the beacon remains close to the image centre.
- Recover after temporary loss using prediction and search/re-acquisition states.
- Expose disturbances representing noise, atmospheric degradation, jitter and platform motion.
- Generate real-time metrics and auditable session reports for benchmark evaluation.

1.2 Solution modes

Mode	Purpose	PTZ behaviour
Simulator	Repeatable virtual environment and stress testing	Virtual camera is actively controlled
External MP4/AVI/MOV	Judge-supplied or prerecorded benchmark footage	PTZ actuation is bypassed; tracking is measured
Live webcam	Host camera experimentation	Image tracking only; device dependent

Design principle: ground truth is available only to the metrics layer. The detector, Kalman estimator and controller operate on image-derived measurements.

2. Problem-Statement Requirements and Traceability

The implementation was tightened against the numerical reference values supplied with SIH26169. The canonical PS-compliant configuration is stored in `scenarios/ps_compliant_reference.json` and is checked by `vcapat/config/ps_compliance.py`.

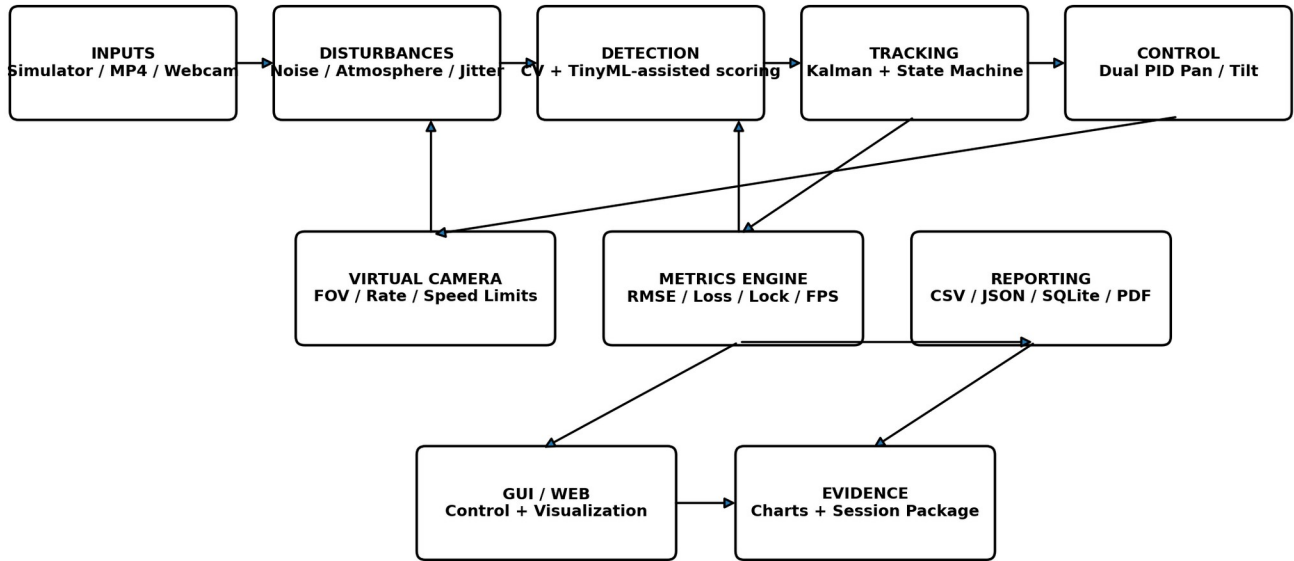
Requirement	Reference target	V-CAPAT PRO implementation	Status
Virtual scene	$\geq 2000 \times 2000$ px	WorldConfig enforces minimum 2000×2000	Implemented
Camera	640×480 suggested; FOV 4×3 deg	Configurable resolution/FOV; reference 640×480 , 4×3	Implemented
Camera rate	≥ 30 Hz	Validated 30-60 Hz	Implemented
Pan / tilt speed	5-10 deg/s; default 5	Validated 5-10 deg/s with command limiting	Implemented
Target size	5-20 px each dimension; default 10×10	Independent width/height, 5-20 px	Implemented
Target motions	Straight, circular, figure-8, random	All four plus spiral/sinusoidal and research presets	Implemented
Noise	Gaussian, Poisson, Salt & Pepper	Selectable/combinable; Gaussian std ≤ 20 px; S&P $\leq 10\%$	Implemented
Jitter	$\leq \pm 20$ px/frame	Bounded configurable camera jitter	Implemented
Atmosphere	Clear, haze, fog, rain, low light	All modes plus optional turbulence warp	Implemented
Platform motion	$\leq \pm 20$ px/frame	Per-frame displacement clamp	Implemented
Performance	Acq ≤ 2 s; error ≤ 10 px; loss $< 5\%$; reacq ≤ 1 s; ≥ 20 FPS	Automatically measured with PASS/FAIL/N/A semantics	Implemented

2.1 Additional software capabilities

The system also provides multi-target designation, multi-algorithm comparison, point-ahead experimentation, sensor-physics approximations, session SQLite storage, stress-test utilities, plugin interfaces, local REST/WebSocket control and a browser companion interface. These additions do not replace the mandatory PS functions; they are separated as extensions.

3. System Architecture

V-CAPAT PRO - Requirement-Driven Coarse Alignment Architecture



Ground truth is isolated to evaluation; detector/tracker/control operate on image data.

Figure 1. End-to-end software architecture. The engine is shared by desktop, browser, headless and benchmark modes.

3.1 Data flow

A frame is produced by the simulator, prerecorded video or webcam. Image-domain disturbances are applied before detection. Candidate regions are detected from the frame and optionally ranked by the TinyML-assisted scorer. The selected image centroid feeds a constant-velocity Kalman filter and lock-state machine. Pixel error relative to the image centre is mapped through the current FOV into angular error and processed by independent pan and tilt PID loops. The virtual camera applies speed/acceleration limits. The metrics subsystem records each frame and evaluates acquisition, tracking, loss, re-acquisition and processing performance.

3.2 Main modules

Subsystem	Primary modules	Responsibility
Simulation	simulation/world.py, camera.py, target.py, trajectories.py	World, beacon generation, motion and virtual camera
Disturbance	disturbance/*	Noise, atmosphere, jitter, platform motion, sensor approximations
Detection	detection/*	Classical CV, centroiding, TinyML-assisted candidate ranking
Tracking	tracking/*, core/state_machine.py	Kalman estimation, association, lock/recovery states
Control	control/*	Dual PID, search patterns, point-ahead and manual assist
Metrics	metrics/*	Per-frame telemetry, compliance, reports, stress evidence
Interfaces	gui/*, api/*	Desktop application, browser control room, REST/WebSocket

4. Configurable Virtual Environment, Camera and Targets

4.1 Virtual world and focal-plane camera

The simulator maintains a world larger than the camera viewport. The reference mission uses a 2000 x 2000 pixel scene. A virtual camera renders a configurable 640 x 480 default viewport with a 4 deg x 3 deg default FOV. Zero pan/tilt corresponds to the scene centre. Update rate is validated at 30-60 Hz and pan/tilt velocity is restricted to the 5-10 deg/s reference range. Optional digital zoom and acceleration limiting are available.

4.2 Beacon model

- One to five simultaneous optical beacon targets.
- Independent target width and height from 5 to 20 pixels; reference target 10 x 10 pixels.
- Square, Gaussian, circular and Airy-like visual profiles.
- Manual, centred or seeded-random initial position.
- Configurable brightness and optional colour representation; monochrome processing remains the default.
- Ground-truth coordinates retained separately for evaluation and never used to steer the tracker.

4.3 Motion library

Motion	Implementation intent
Straight line	Constant-direction movement with boundary handling
Circular	Periodic angular motion around a configured centre
Figure-8	Two-axis periodic crossing trajectory
Random	Seeded stochastic movement for repeatability
Spiral / Sinusoidal	Optional additional benchmark patterns
LEO / UAV / HAPS / Aircraft / Star	Illustrative kinematic visual presets; not claimed as measured mission ephemerides



Figure 2. Example locked simulator frame from the development validation run.

5. Disturbance and Environmental Modelling

The disturbance chain is intentionally configurable so the same tracking algorithm can be exercised under repeatable changes in visibility and platform behaviour. The models are software-domain approximations intended for algorithm development; they are not a substitute for calibrated atmospheric propagation or laboratory optics.

Disturbance	Controls / limits	Purpose
Gaussian noise	std 0-20 px	Random sensor/image noise
Poisson noise	selectable	Photon-count style intensity variation
Salt & Pepper	0-10% occupancy	Impulse/corruption robustness
Camera jitter	0 to +/-20 px/frame	Platform/camera vibration proxy
Atmosphere	clear, haze, fog, rain, low-light	Visibility/contrast degradation
Turbulence warp	optional, configurable Fried parameter	Spatial distortion experiment
Contrast / brightness	operator adjustable	Explicit image degradation control
Platform motion	linear/circular/sinusoidal/random/figure-8; step <=20 px/frame	External platform motion proxy
Occlusion	scenario driven	Loss/coasting/re-acquisition testing
Sensor physics	QE/read/dark noise/exposure/bit-depth/hot pixels	Optional FPA-style engineering approximation

5.1 Reproducibility

Scenarios carry a random seed. Replaying the same configuration and seed reproduces the same simulator target/noise sequence within the software model, which supports fair algorithm comparisons and repeatable demonstrations.

5.2 Honesty of modelling scope

Atmospheric and sensor effects are image-domain or simplified engineering models. V-CAPAT PRO does not claim calibrated wave-optics propagation, scintillation certification, measured detector quantum-efficiency curves or flight qualification.

6. Automatic Beacon Detection and AI-Assisted Ranking

6.1 Classical computer-vision paths

The detection arena provides four independent image-processing paths: adaptive thresholding, morphological top-hat processing, template-based thresholding and optical flow. Connected components are evaluated using local brightness, geometry and track proximity. Weighted centroiding provides subpixel-like centroid refinement where intensity structure allows it.

6.2 TinyML-assisted candidate model

Version 1.1 includes a compact logistic classifier used in `tinyml` and `ai_auto` modes. It is trained deterministically at startup from synthetic candidate-feature distributions. Features include normalized brightness, compactness, aspect symmetry, plausible component area and distance from the predicted target location. The learned probability is fused with the interpretable candidate score.

Appropriate claim: "AI-assisted lightweight beacon candidate classification combined with explainable computer vision." The model is not presented as a deep neural network trained on real ISRO FSOC imagery.

6.3 Explainability and fallback

- Candidate score contributions are available for debugging/X-Ray views.
- Classical detection remains available if the learned candidate ranker is disabled.
- An ONNX/YOLO adapter exists for future trained weights, but no unverified beacon model is bundled.
- The detector never reads simulator ground-truth coordinates for target selection.

6.4 Automatic designation

For the mandatory single-target case, the highest-confidence valid candidate is associated with the active track. Multi-target extensions provide explicit designation and optional handover logic for research scenarios.

7. State Estimation, Acquisition and Pan-Tilt Control

7.1 Kalman state estimation

The tracker uses a constant-velocity state vector $[x, y, vx, vy]$. Prediction is performed every frame and measurement updates occur when a valid beacon candidate is associated. This reduces centroid noise and provides short-term motion prediction during brief detection gaps.

7.2 Lock-state machine

State	Meaning	Control behaviour
SEARCHING	No reliable target track	Search pattern / scene scan
ACQUIRING	Candidate observed but lock not yet confirmed	Prediction and bounded centring
LOCKED	Stable designated track	Closed-loop PID centring
COASTING	Short temporary detection gap	Kalman prediction maintains continuity
REACQUIRING	Loss threshold exceeded	Wider search/reassociation logic

7.3 Dual PID virtual pan-tilt control

The controller computes horizontal and vertical pixel error relative to the image centre. FOV and resolution convert pixel displacement to angular error. Independent PID loops generate pan and tilt velocity commands. Anti-windup, deadband and configured speed limits protect the simulated mechanism. Optional acceleration limiting prevents unrealistic instantaneous velocity changes.

7.4 External video behaviour

When a prerecorded benchmark is processed, the application deliberately bypasses simulated PTZ actuation. A prerecorded frame cannot be physically re-centred after capture; therefore detection/tracking quality is evaluated against labelled ground truth while keeping the control-path distinction explicit.

8. Metrics, Performance Logs and Evaluation Criteria

Every processed frame records source mode, timestamps, ground truth when available, detected/filtered coordinates, confidence, lock state, centroid and pointing errors, camera pan/tilt, processing time, instantaneous FPS and disturbance metadata. The session summary derives the metrics required by the problem statement.

Metric	Definition in V-CAPAT PRO	Acceptance target
Acquisition time	Time to first confirmed LOCKED state	≤ 2 s
Tracking error	Pointing RMSE in simulator; centroid RMSE for labelled fixed-camera video	≤ 10 px
Target loss	Percent of visible frames without valid lock/detection evidence	$< 5\%$
Re-acquisition	Measured loss-to-recovered-lock event duration	≤ 1 s
Processing rate	$1000 / \text{average processing milliseconds}$	≥ 20 FPS
Lock retention	Percent of visible frames retained in lock	Reported
Processing time	Average and maximum pipeline milliseconds	Reported
Error extrema	Mean/RMSE/maximum centroid or pointing error	Reported

8.1 Evidence outputs

- frames.csv - frame-by-frame telemetry
- summary.json and compliance.json - summary metrics and PASS/FAIL/N/A checks
- events.json - acquisition/loss/recovery events
- config_snapshot.json - exact scenario settings
- frames.sqlite - queryable time-series database
- charts/*.png - error/FPS/pan-tilt/link plots
- certificate.pdf - one-page software evidence summary
- optional annotated MP4 - visual audit trail

N/A is never silently counted as PASS. For example, a run with no target loss has no measured re-acquisition event and therefore reports re-acquisition as N/A.

9. Application Interfaces and Benchmark Workflow

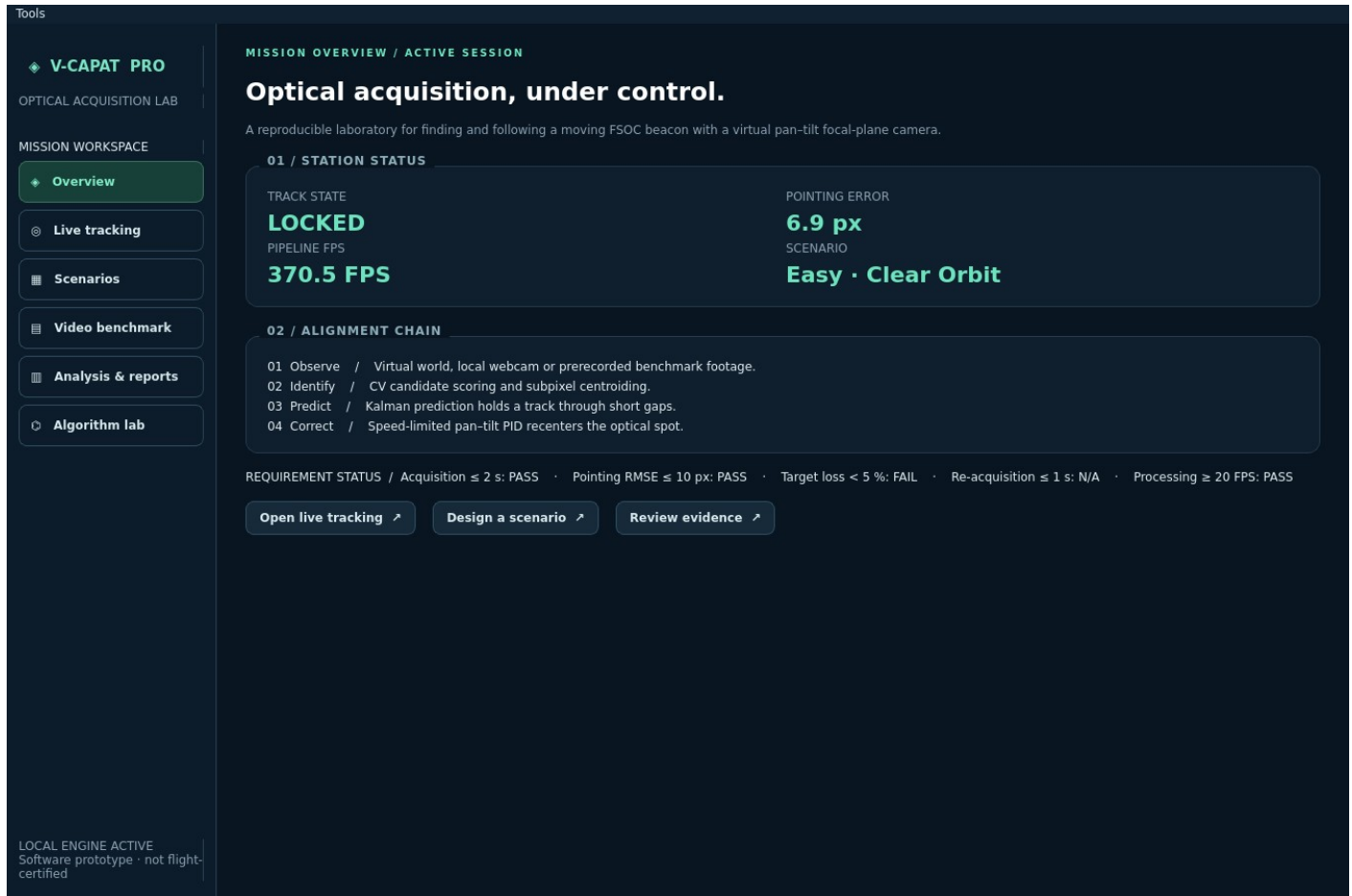


Figure 3. V-CAPAT PRO browser control room / project visual interface preview.

9.1 Desktop application

The PySide6 desktop interface exposes live tracking, scenario selection, camera and target configuration, disturbances, detector selection, control/tracking tuning, reports, stress tests and analysis tools. The reference constraints are validated before a scenario is accepted.

9.2 Browser companion control room

A six-page local control room provides Overview, Live Tracking, Scenarios, Video Benchmark, Analysis & Reports and Algorithm Lab views using the same shared engine. Version 1.1 exposes AI-assisted auto and TinyML detector selections in the browser as well as the desktop.

9.3 Judge benchmark path

The evaluator can open an MP4/AVI/MOV file. If a CSV with frame, timestamp_s, gt_x, gt_y and visible columns is supplied, the application measures centroid error and lock metrics against that file. The sample benchmark included under assets demonstrates the complete path.

10. Validation Methodology and Representative Results

Validation is layered: (1) unit/integration tests, (2) PS configuration/smoke validation, (3) deterministic scenario runs, and (4) labelled external-video processing. Performance values below are representative development-environment measurements from this codebase on 02 October 2026. They are not guaranteed on different hardware and are not flight-qualification results.

Run	Acq (s)	Error metric (px)	Loss (%)	Re-acq (s)	Proc. FPS	Result
Easy - Clear Orbit	0.1	Pointing RMSE 5.724	0.67	N/A	407.08	All measured checks PASS
Emergency Maneuver	0.1	Pointing RMSE 9.323	0.74	0.867	83.36	All five checks PASS
Labelled sample MP4	0.1	Centroid RMSE 0.121	1.67	N/A	353.91	Measured video checks PASS

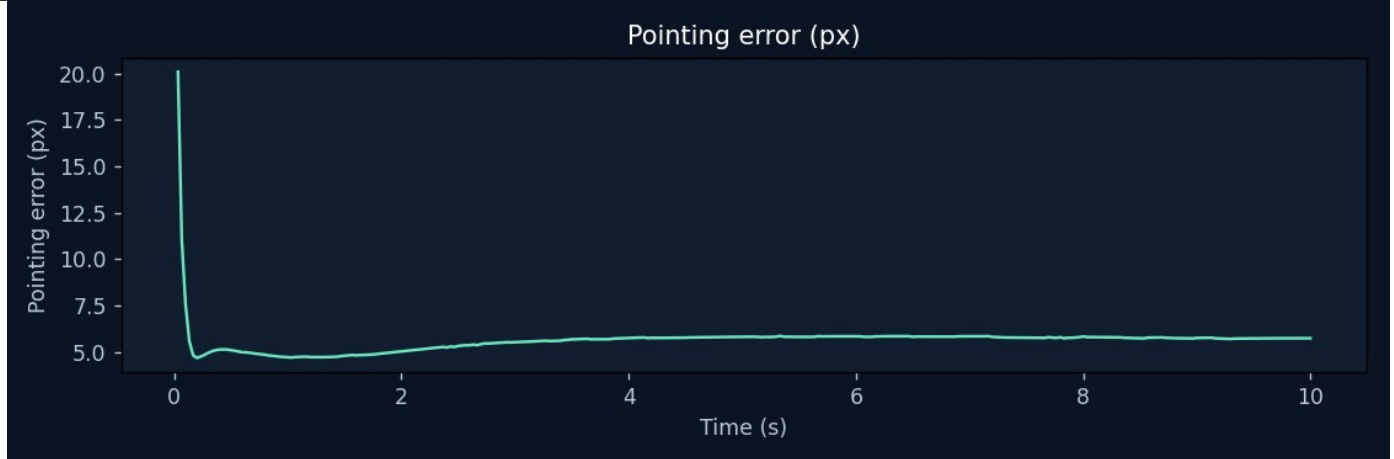


Figure 4. Easy clear-orbit pointing error across the 10 s validation run.

10.1 Automated test coverage

The source project includes tests for simulator configuration, target motion, disturbance/physics integration, tracking recovery, multi-target features, reports, plugin loading, web/API behaviour, compliance helpers and release-facing functionality. The Windows release pipeline installs all declared dependencies, runs the complete test suite, runs `validate_mvp.py`, then performs a packaged executable self-test.

11. Re-acquisition Stress Evidence

The Emergency Maneuver scenario is used because it contains an actual loss and recovery event; this avoids claiming re-acquisition performance from a run in which the target was never lost.

Measure	Observed	PS target	Assessment
Acquisition time	0.1 s	≤ 2 s	PASS
Pointing RMSE	9.323 px	≤ 10 px	PASS
Target loss	0.74 %	< 5 %	PASS
Maximum re-acquisition	0.867 s	≤ 1 s	PASS
Average processing rate	83.36 FPS	≥ 20 FPS	PASS



Figure 5. Emergency scenario pointing error. The large excursion corresponds to the forced loss/manoeuvre followed by recovery.



Figure 6. Final frame after recovery: target is re-centred and tracker state is LOCKED.

12. Standalone Windows Executable and Release Engineering

The mandatory standalone deliverable is addressed in v1.1 with a reproducible Windows release gate rather than an unverified PyInstaller command. The repository contains both an onedir and onefile build specification, Windows version metadata, packaged resource-path handling, executable self-test logic, SHA-256 release checksums and a GitHub Actions Windows builder.

Windows Standalone Release Gate



A release is packaged only after tests, validation and the built executable itself complete the smoke test.

Figure 7. Release pipeline. Packaging is stopped if automated tests, PS validation or the built executable smoke test fails.

12.1 Primary build

Running `build_release.bat` builds and verifies both formats. The recommended SIH artifact is `release/VCAPAT_PRO_v1.1.0_Windows_x64.zip`, containing `VCAPAT_PRO.exe` and its runtime files plus a `submission_docs` folder. A onefile ZIP is generated as a secondary convenience build.

12.2 Packaged executable self-test

`VCAPAT_PRO.exe --self-test` checks that bundled scenarios, assets, sample benchmark data and this technical report are available from the frozen resource path. It then processes at least 30 frames and verifies that CSV/JSON/PDF evidence can be exported. The Windows build script runs this command against the freshly built executable before packaging the release.

12.3 CI option

The included `.github/workflows/windows-release.yml` workflow builds on GitHub windows-latest, runs the same release gate and uploads the verified release ZIPs. This provides a reproducible Windows build path even when a development machine does not have a clean packaging environment.

A Windows executable must still be built on Windows (or the provided Windows CI runner). This report does not falsely claim that a Linux-built binary is a verified Windows executable.

13. Limitations, Future Work and Conclusion

13.1 Current limitations

- Atmospheric turbulence and optical link effects are software approximations, not calibrated propagation through a measured channel.
- The TinyML classifier uses synthetic candidate-feature training data; a mission-quality detector needs representative labelled optical imagery.
- Webcam performance and final Windows GUI behaviour depend on the target demo machine and camera drivers.
- Synthetic LEO/UAV/HAPS/aircraft trajectories are visual/kinematic presets unless external ephemeris or measured data are explicitly supplied.
- Hardware-in-the-loop PTZ and real optical sensor calibration remain future integration tasks.

13.2 Future improvements

- Train and validate a compact neural beacon detector on representative real/synthetic optical datasets.
- Add calibrated wave-optics/turbulence propagation and measured FPA response models.
- Integrate real PTZ hardware through the existing controller/plugin boundary.
- Validate performance across multiple machines and long-duration sessions.
- Add signed Windows releases and organization-specific code-signing when a certificate is available.

13.3 Conclusion

V-CAPAT PRO implements the complete software coarse-alignment loop requested by SIH26169: configurable virtual environment, moving beacon generation, automatic acquisition, image-based position estimation, continuous tracking, closed-loop virtual pan-tilt control, disturbances, recovery states and real-time/reportable performance metrics. Version 1.1 additionally makes the two mandatory submission risks explicit and actionable: a formal 10-15 page technical report is delivered with the source, and the Windows standalone package is generated through a test-gated, self-verifying build pipeline.

References

1. Smart India Hackathon 2026, Problem Statement SIH26169: AI-Based Virtual Camera Tracking System for Coarse Alignment of Mobile FSOC Terminals.
2. ITU-R Recommendation P.1817-1, propagation data required for the design of terrestrial free-space optical links.
3. H. Kaushal and G. Kaddoum (2017), Free Space Optical Communication: Challenges and Mitigation Techniques, IEEE Communications Surveys & Tutorials.
4. NASA/JPL acquisition, tracking and pointing literature referenced in the project research slide (NASA NTRS citation 20060040875).
5. R. E. Kalman (1960), A New Approach to Linear Filtering and Prediction Problems.